

1. Create an array of student objects (``name`, `dept`, `year``). Display them using ``.map()`` inside cards.
2. Show “Welcome back!” if a variable ``isLoggedIn`` is true, else show “Please log in”.
3. Create a `<ProfileCard>` component that takes props: ``name`, `role`, and `image``. Use it three times with different data.
4. Create an input field for “Username.” When the user types, display the entered name live below it.
5. Create a small form with “Name” and “Email” fields. On submit, display both values below without refreshing the page.
6. Display an array of product objects (id, name, price) in a `<table>` using ``.map()``.
7. Create a parent component with a function ``showAlert()``. Pass it as a prop to a child and call it on button click.
8. Given a list of products (name, category), add buttons like “All”, “Electronics”, “Clothes” to filter and display accordingly.
9. Create a search bar that filters and displays matching student names from an array as you type.
10. Import a local ``data.json`` file (list of products) and display them as cards using ``.map()``.
11. Fetch data from ``https://dummyjson.com/products`` and display product name, price, and brand.
12. Show “Loading...” while fetching, and if the request fails, show “Error loading data”.
13. Start with an array of fruits in state. Add a form to insert new fruits dynamically into the list.
14. Extend the previous question — add a delete button to remove an item from the list.
15. Create a profile object in state (``name`, `email`, `phone``). Add inputs to edit and update each field.
16. Create a form (title, body) and send data to ``https://dummyjson.com/posts/add`` using Axios POST. Display “Post Added!” message on success.
17. Fetch one user from ``https://dummyjson.com/users/1``, display details in inputs, edit and update using PUT.
18. Fetch 20 users from an API and show only 5 at a time with “Next” and “Prev” buttons.
19. Build a 3-page app — Home, About, Products — and link them using ``react-router-dom``.
20. Add a “Products” page with nested routes: ``/products/electronics`` and ``/products/fashion``. Display relevant text for each.